

Speaking GitHub Copilot's Language

Context engineering for gigantic codebases



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Don't bring the monorepo to the model.

Bring the model to the right neighborhood.

Context is a finite budget, not a landfill.



Exercise files

github.com/timothywarner-org/eShop



Instead of stuffing context into your LLM...

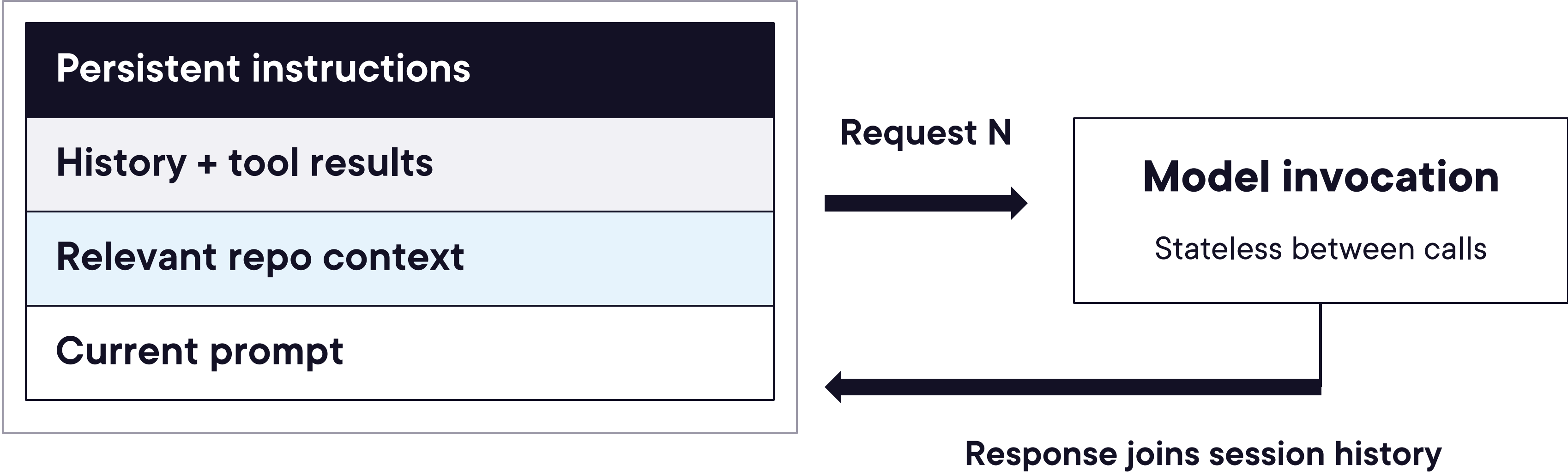


...sculpt your LLM around your context



The session holds the state

Stateful Copilot session



Next call: assemble a fresh request from relevant state.

System instructions, tool definitions, and editor context also use the budget.



The payload grows. The budget doesn't

One finite context window per model request

Early



Later



After
compaction



Keep the goal and decisions. Re-read exact evidence when needed.

Same task: compact history. Different task: start a fresh session.

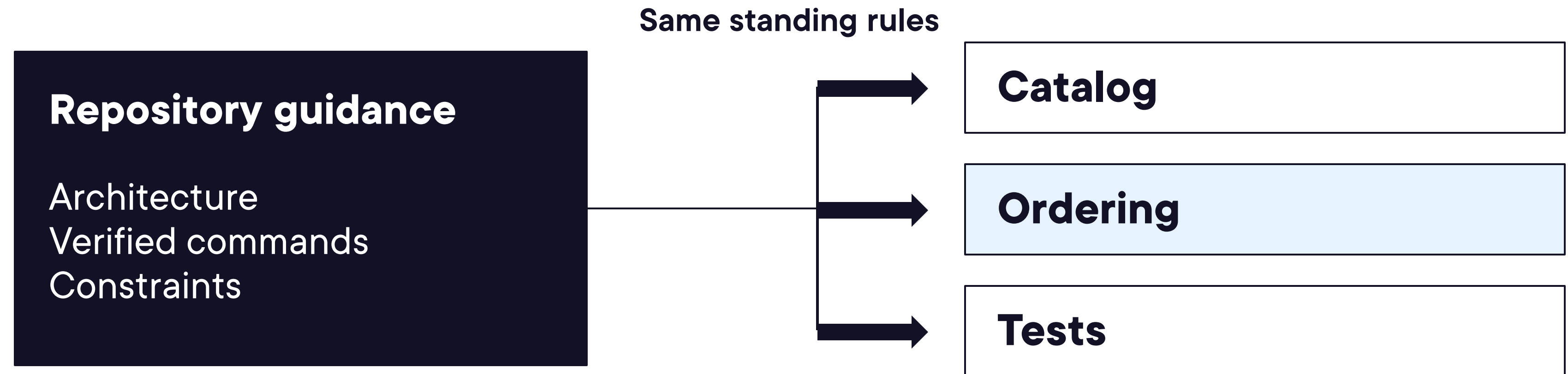
Conceptual proportions. Compaction summarizes older history and can omit detail.



Standing guidance applies across the repo

.github/copilot-instructions.md

Short, stable, actionable. Always-on for Copilot Chat.



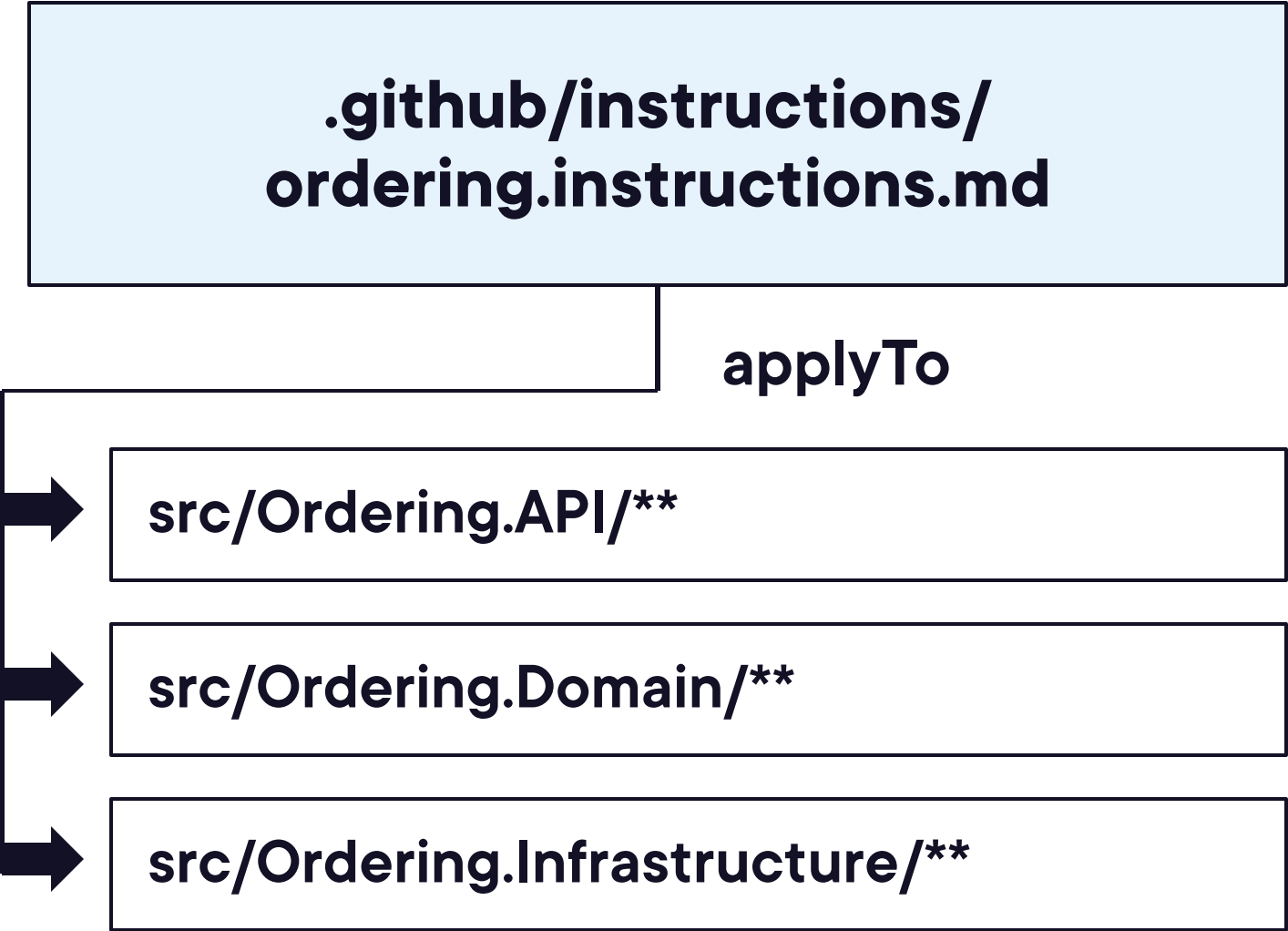
Copilot CLI: /init

Creates or updates the file. Review it.



One source of truth per rule

Mechanism	Scope
copilot-instructions.md	Repository
*.instructions.md	Matching paths
AGENTS.md	Optional portability



Directory scoping with AGENTS.md depends on the client.

In VS Code, nested AGENTS.md support is experimental.

Add a rule where it belongs. Avoid mirrored instructions.



Retrieval selects the working set

Task: trace the buyer and payment IDs



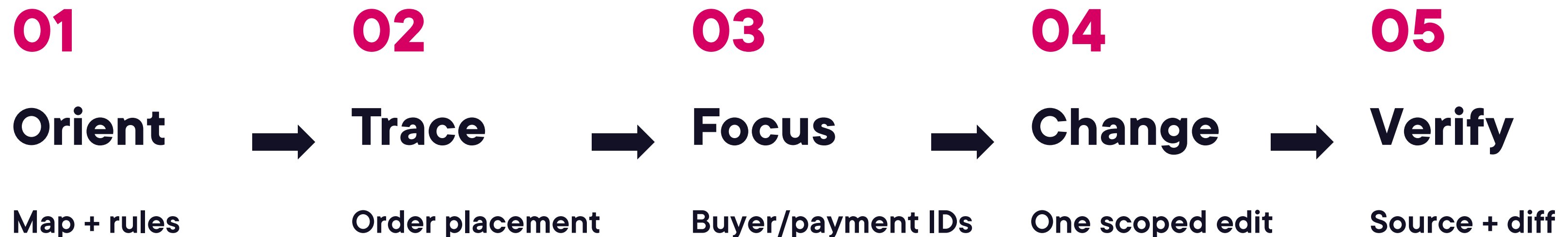
Semantic index: find code by meaning. Source: check the claim.

Check index status. Search and file reads can still work without it.



Now: one eShop flow in VS Code

[C:\github\eshop-1](#)



Where do Buyer.Id and Payment.Id reach the order?

Cite the path. Check the claim. Preserve the useful rule.



Thank you for your time!

Tim Warner | TechTrainerTim.com

